🤖👨🏻‍🍳 ChefBot: AI Recipe Chatbot

🥑🥦🥕 Ingredients (comma-separated):

Select a cuisine:

Any

Generate Recipe

🍽️ Generated Recipe:

Recipe:

Ingredients:
- 1 lb boneless, skinless chicken breasts
- 2 cloves garlic, minced
- 1 can (28 oz) diced tomatoes
- 1 tbsp tomato paste
- Salt and pepper to taste

Instructions:
1. Preheat the oven to 375°F (190°C).

💾 Save Recipe

🔍 Find Alternative Ingredients

Enter an ingredient:

Select Search Method:

- (•) Annoy (Fastest)
- ( ) Direct Search (Best Accuracy)

🔄 Find Alternatives

HW3 Objectives

In this assignment, you will **modify and enhance** a working **recipe generation app** powered by **Hugging Face Transformers, SpaCy, and Annoy**. This hands-on homework will help you explore **prompt engineering, text generation strategies, efficient similarity search, and app extensibility**. You'll gain hands-on experience with LLM inference and vector-search.

Working with **Streamlit** is simple. For a quick tutorial check this out:
https://docs.streamlit.io/get-started/fundamentals/main-concepts

By the end of this assignment, you will:

- **Experiment with text generation parameters** (temperature, sampling, decoding) to see how they impact AI responses.
- **Modify prompts** to control response **structure, detail level, and creativity**.
- **Implement an ingredient substitution feature** using both **direct cosine similarity search** and **Approximate Nearest Neighbors (ANN) with Annoy** to improve search speed.
- **Extend the app** by adding **a custom feature of your choice**, demonstrating creativity and technical skills.

📌 Understanding the Provided Code

You have been given a **working recipe generation app** (Recipe_Bot.py). It includes:

- **Recipe generation** using one of the following language model.
- **Cuisine selection** (e.g., Italian, Chinese, Indian, Mexican, etc.).
- **Saving recipes** to a user's collection.
- **UI for an alternative ingredient finder is implemented,** but the functionality is missing (you will complete it in Task 3 and Task 4).

Some of the best LLMs under ~1B parameters:

- **Gemma-3-270m-it**
  - https://huggingface.co/google/gemma-3-270m-it
- **SmolLM2-360M-Instruct**
  - https://huggingface.co/HuggingFaceTB/SmolLM2-360M-Instruct
- **Qwen2.5-0.5B-Instruct**
  - https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct
- **Qwen3-0.6B**
  - https://huggingface.co/Qwen/Qwen3-0.6B
- **Llama-3.2-1B-Instruct**
  - https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct

Your goal is to **improve and extend** this application by completing the assigned tasks.

☃️ Have fun building your first Gen-AI app! ☃️

Task 1: Experimenting with Text Generation Parameters

Objective: Modify text generation parameters to understand their impact on AI-generated recipes.

📖 Background

When generating text, LLMs use several parameters that control randomness, diversity, and quality. Here, we will experiment with **three key aspects**:

1. **Temperature Scaling**: Controls randomness.
2. **Sampling Strategies**:
   - top_k: Limits the number of possible next words.
   - top_p: Uses nucleus sampling (focuses on high-probability words).
3. **Decoding Strategies**:
   - **Greedy Decoding** (default, picks highest probability word each step).
   - **Beam Search** (considers multiple paths before finalizing output).

📌 **Read:** Hugging Face Generation Strategies Documentation
🔗 https://huggingface.co/docs/transformers/en/generation_strategies

🔷 Steps:

1. Locate the function in Recipe_Bot.py that generates text using model.generate().
2. Modify it to **expose the following parameters**:
   - **Temperature**: Set to {0.5, 1.0, 2.0} and compare outputs.
   - **Sampling (do_sample=True)**: Try different top_k values {5, 50} and top_p values {0.7, 0.95}.
   - **Decoding strategy**: Compare **default greedy decoding** (num_beams=1) with **beam search** (num_beams=5).
3. **Generate a recipe** using different settings and **compare outputs**.

🔍 What to observe?

- How does increasing *temperature* affect creativity/randomness?
- How do *top_k* and *top_p* change the diversity of words?
- Does *beam search* produce better-structured recipes than greedy decoding?

📤 Deliverable:

- Provide some screenshots of the output with different parameter settings.
- A concise, approximately **150 words,** discussion explaining observed differences.

Task 2: Controlling AI Responses through System Prompts

Objective: Modify the **system prompt** to control **format, detail level, and creativity** in the AI-generated recipes.

📖 Background

LLMs respond differently based on how you phrase prompts. Your task is to **experiment with system prompts to shape the model's output.**

Instead of just modifying tone, we will explore **three distinct controls**:

1. **Controlling response format**:
   - Example: Ensure the response **always** includes a **title, ingredients, and step-by-step instructions** in that order.
   - Example prompt: "Generate a recipe. First, give a title. Then list ingredients. Then provide step-by-step instructions."
2. **Adjusting response length and detail**:
   - Example: A **brief** vs. **highly detailed** recipe.
3. **Encouraging creative or unconventional recipes**:
   - Example: **Surprise users with unique combinations.**

🔷 To Do:

1. **Find the system prompt** in **Recipe_Bot.py**
2. Modify it **three different ways**:
   a. **Control format** (structured output with title, ingredients, steps).
   b. **Vary detail level** (concise vs. detailed).
   c. **Encourage creativity** (unusual ingredient pairings).
3. Generate recipes using each variation and **compare outputs**.

🔍 What to observe?

- Does the model follow structured instructions?
- Does length control work as expected?
- How creative does the model get when asked?

📤 Deliverable:

- Provide some screenshots of the output by your final three system prompts.
- A concise, approximately **150 words,** discussion explaining observed differences.

Task 3: A Simple Alternative Ingredient Finder (Direct Search)

Objective: Add a feature that suggests **alternative ingredients** using **direct search with cosine similarity**.

📖 Background

When users enter an ingredient, we want to suggest **similar alternatives**. The simplest approach is:

- **Compare input ingredient with every other ingredient** in ingredient_list using **cosine similarity**.
- **Return the top-3 most similar ingredients**.

✅ **Good News:**

- To make this easier, we have already provided a **cosine similarity function** in the code. You can use it to compute similarity scores efficiently.
- Additionally, the UI for this feature is **already implemented** in **Recipe_Bot.py**.

⭕ **Your task is to complete** the function: direct_search_alternatives(ingredient):

🔷 To Do:

Step 0: Locate the function direct_search_alternatives(ingredient): in **Recipe_Bot.py**
Step 1: Retrieve the vector embedding for the input ingredient.
Step 2: Iterate through all ingredients in ingredient_list and compute cosine similarity.
Step 3: Sort the ingredients by similarity score in descending order.
Step 4: Return the **top-3 most similar** ingredients.

📌 **Example:**

- **Input:** "Milk"
- **Output:** "Soy Milk ⟶ Almond Milk ⟶ Coconut Milk"

🔍 What to Observe?

- How **accurate** are the suggested alternatives? Do this similarity **capture meaning** behind the words?
- How **fast/slow** is this method?

📤 Deliverable:

- Updated direct_search_alternatives(ingredient) function.
- A concise, approximately **150 words,** summary of your observations along with Screenshot of the working feature.

Task 4: Faster Alternative Ingredient Finder (ANN by Annoy)

Objective: Improve search speed and user experience by using **Approximate Nearest Neighbors (ANN)** with **Annoy**.

📖 Background

Have you noticed how **slow** direct search in Task 3 was?
This is because **comparing every ingredient one-by-one takes too long**. A better way?
Approximate Nearest Neighbors (ANN) using Annoy by Spotify 🎶🎶**!** 🚀

Instead of searching through all ingredients directly, **Annoy** pre-builds an efficient search index, allowing us to retrieve **nearest neighbors instantly**.

📌 **Read:** Annoy Documentation
🔗 https://github.com/spotify/annoy

✅ **Good News #1:** The UI for this feature is **already implemented** in **Recipe_Bot.py**
✅ **Good News #2:** The function **build_annoy_index() is already provided**!

🛠️ What is build_annoy_index() and How to Use It?

- This function **builds the Annoy index** by storing **vector representations of all ingredients (considering their relationships)** in an **searchable format**.
- It is **cached** using @st.cache_resource, meaning it only needs to be computed once.
- Instead of manually iterating through all ingredients, you can **use the index to find nearest neighbors efficiently**.

📌 How to use it?

- Call build_annoy_index() to **get the pre-built index**.

⭕ **Your task is to complete** the function annoy_search_alternatives(ingredient)

🔷 To Do:

- Step 0: Locate the function annoy_search_alternatives(ingredient): in **Recipe_Bot.py**
- Step 1: Get the **pre-built Annoy index** using build_annoy_index().
- Step 2: Get the vector embedding for the input ingredient.
- Step 3: Use **Annoy's nearest neighbor search** to find the **top-3 most similar** ingredients.
- Step 4: Return the **top-3 nearest neighbors**.

📌 Example:

- **Input:** "Milk"
  - **Direct Search (Task 3):** "Soy Milk ⟶ Almond Milk ⟶ Coconut Milk"
  - **Annoy (Task 4):** "Oat Milk ⟶ Soy Milk ⟶ Cashew Milk"

Note: This is just an example output, you might get something different, that's perfectly fine as long as they make sense!

🔍 What to Observe?

- How **accurate** are the suggested alternatives? Do this similarity **capture meaning** behind the words?
- How **fast/slow** is this method?

📤 Deliverable:

- Updated annoy_search_alternatives(ingredient) function.
- A concise, approximately **150 words,** summary of your observations along with Screenshot of the working feature.

Task 5: Let's add ONE new AI-powered Feature of Your Choice

Objective: Extend the app with a **new feature** of your choosing.

This task allows you to **be creative** and add an enhancement that improves user experience or makes the app more interactive. You can choose any meaningful feature that complements the existing recipe generator.

🔷 Example Ideas (Just to give you a sense, please come up with your OWN ORIGINAL ideas)

1. **Recipe Text-to-Speech** 🎙️
   - Convert the generated recipe into **spoken audio** using a text-to-speech library.
   - Allow users to **listen** to the recipe instead of reading it.
2. **Recipe Personalization** 👤
   - Adjust recipes based on user preferences.
   - Example: Users enter their **dietary preferences (e.g., low-carb, high-protein, vegan)**, and the app **adapts ingredient choices** accordingly.

These are just suggestions—**you can come up with your own innovative feature!** 🚀

📤 Deliverable:

- A **short description** of the new feature along with a **Screenshot** demonstrating the feature in action
- **Updated code** including the implementation of this new feature.

📌 Grading Guidelines

Your submission for Part 5 will be evaluated based on the following:

Criteria	Weight	Expectations
Functionality	40%	The feature works correctly, integrates well, and improves the app.
Creativity & Complexity	30%	Unique ideas and technically interesting implementations will be appreciated.
Clarity & Documentation	30%	Code should be clean and well-commented, with clear comments, brief explanation, and screenshots of the output.

Bonus: Exceptional effort, extra polish, or an innovative feature may earn additional credit! 🌟

📤 Final Submission

You should submit two files:

✅ **Modified code (final version)** with all changes, including:
- The two **alternative search methods** (direct search and ANN with Annoy).
- The **new feature** implemented in Task 5.

✅ A **PDF report** including:
- All deliverables for the 5 tasks. Including, observations, justifications and screenshots of results.

Note: Please rename your submission files as follows:
- **Python File: RecipeBot_YourName.py** (Replace **YourName** with your actual name).
- **PDF Report: HW3_YourName.pdf** (Replace **YourName** with your actual name).

Important Note: Report must be written by YOU, AI-generated report receives 0.

Environment Setup Guide for HW3 - Generative AI Course

Step 1: Ensure You Have Python 3 Installed

Before proceeding, make sure you have **Python 3.9 or later** installed.
Check your Python version by opening terminal and running:

```
python --version
```

or

```
python3 –version
```

If you don't have Python 3 installed, install it first or upgrade it to 3.9 or later.

Step 2: Create a Virtual Environment (Recommended)

It's best to use a virtual environment to manage dependencies and avoid conflicts.

1. Open a terminal (or command prompt) and navigate to your project directory.
2. Run the following command to create a virtual environment:
   ```
   python -m venv genai_hw3_env
   ```
3. Activate the virtual environment:
   - **Windows (Command Prompt)**:
     ```
     genai_hw3_env\Scripts\activate
     ```
   - **Windows (PowerShell)**:
     ```
     .\genai_hw3_env\Scripts\Activate.ps1
     ```
   - **Mac/Linux**:
     ```
     source genai_hw3_env/bin/activate
     ```

Step 3: Upgrade Pip

Ensure pip is up to date:

```
pip install --upgrade pip
```

Step 4: Install Required Packages

You didnt'Use the provided requirements.txt file to install dependencies:

1. Place the requirements.txt file in your project directory.
2. Run the following command:
   ```
   pip install -r requirements.txt
   ```

Step 5: Verify Installation

Check if packages are installed correctly:

```
python -c "import streamlit, pandas, spacy, numpy, torch, transformers; print('All packages installed successfully!')"
```

Step 6: Run Streamlit App

To start your **Recipe Bot** application, run:

```
streamlit run Recipe_Bot.py
```

This will launch the app in your web browser. 🚀